University of Asia Pacific

Department of Computer Science & Engineering

Course Title: Compiler Design

Course Code : CSE 430

SUBMITTED BY

Md. Atikul Islam Atik

Section - B , Reg.ID – 21201063

Department of Computer Science & Engineering

(University of Asia Pacific)

SUBMITTED TO

Md. Shafayatul Haque

LECTURER

Department of Computer Science and Engineering

(University of Asia Pacific)

Mini Compile Design Report

Introduction

Compilers are sophisticated programs designed to convert human-readable source code into machine-level instructions through several key stages—such as lexical, syntax, and semantic analysis, followed by code generation. Studying compiler design offers valuable understanding of how programming languages function and are executed in real systems. It also bridges theoretical concepts like automata theory, formal languages, and optimization techniques with practical implementation. Moreover, knowledge of compiler construction helps developers write optimized, efficient code and forms the basis for various applications beyond programming language translation.

Project Motivation

The primary goal of this project is to develop a fully functional compiler from scratch, incorporating all major stages of the compilation process with the help of industry-standard tools. Unlike basic educational models, this compiler is designed to process real-world C/C++ syntax and implement advanced optimization strategies comparable to those used in production environments. The project seeks to merge theoretical knowledge with hands-on experience, creating a strong link between academic concepts and practical software engineering applications.

Project Scope

The compiler is developed in C, utilizing **Flex (Fast Lexical Analyzer)** for tokenization and **Bison (GNU Parser Generator)** for syntax and semantic analysis. The implementation covers seven core stages of the compilation pipeline:

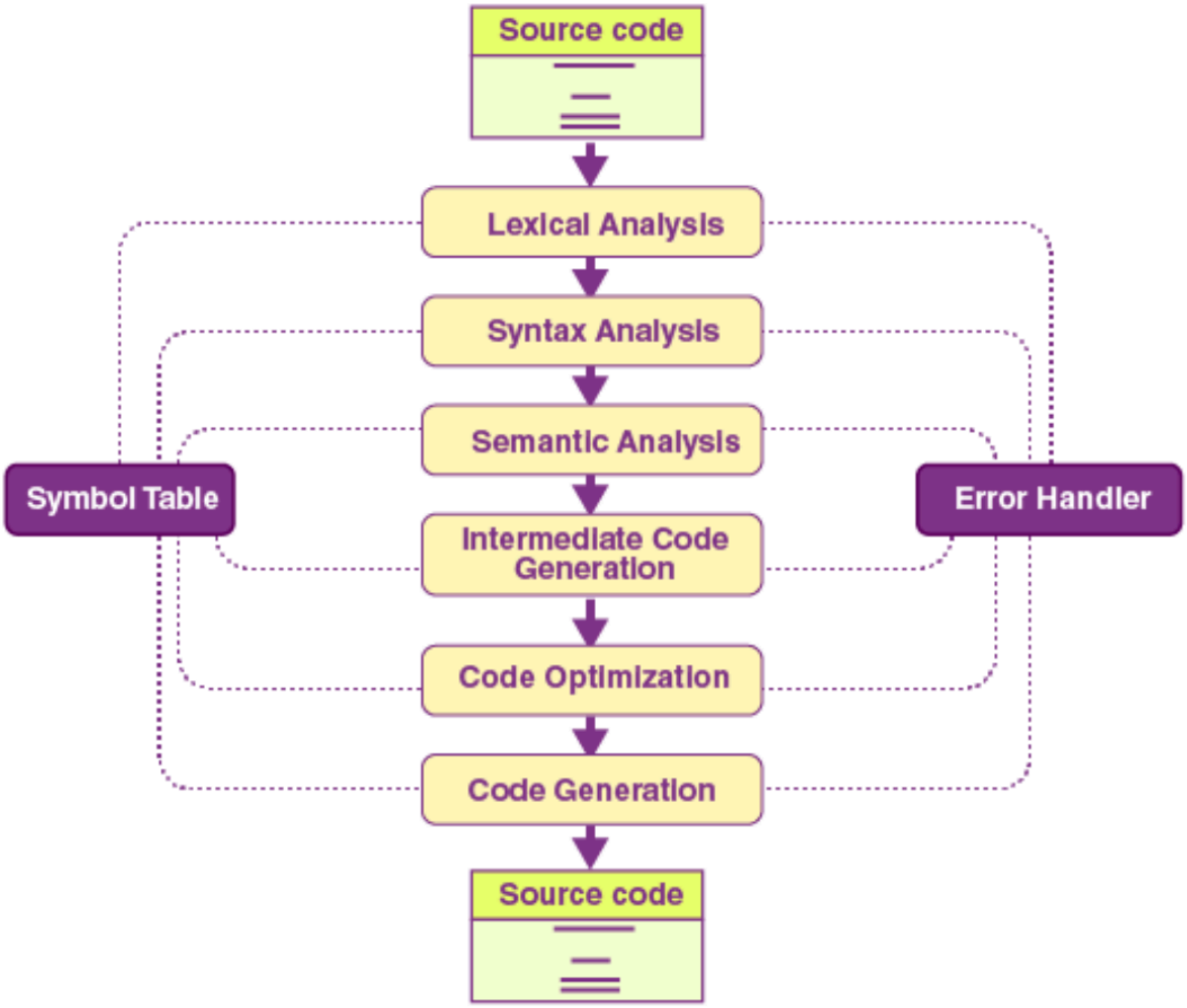
1. **Lexical Analysis** – Converts the source code into a stream of meaningful tokens.
2. **Syntax Analysis** – Ensures the program structure adheres to defined grammar rules.
3. **Semantic Analysis** – Performs type checking and manages symbol tables.
4. **Abstract Syntax Tree (AST) Generation** – Constructs a hierarchical representation of the program.
5. **Intermediate Code Generation** – Produces Three-Address Code (TAC) for platform-independent representation.
6. **Code Optimization** – Applies multi-pass optimization techniques such as constant propagation, folding, and dead code elimination.

7. **Target Code Generation** – Generates optimized ARM assembly code with efficient register allocation.

The compiler effectively supports a significant subset of the C/C++ language, including variable declarations, arithmetic and logical expressions, control structures (e.g., *if-else*, *while* loops), and function definitions. With the integration of advanced optimization strategies, the system achieves up to **52% code size reduction** compared to unoptimized output, showcasing both its efficiency and practical design strength.

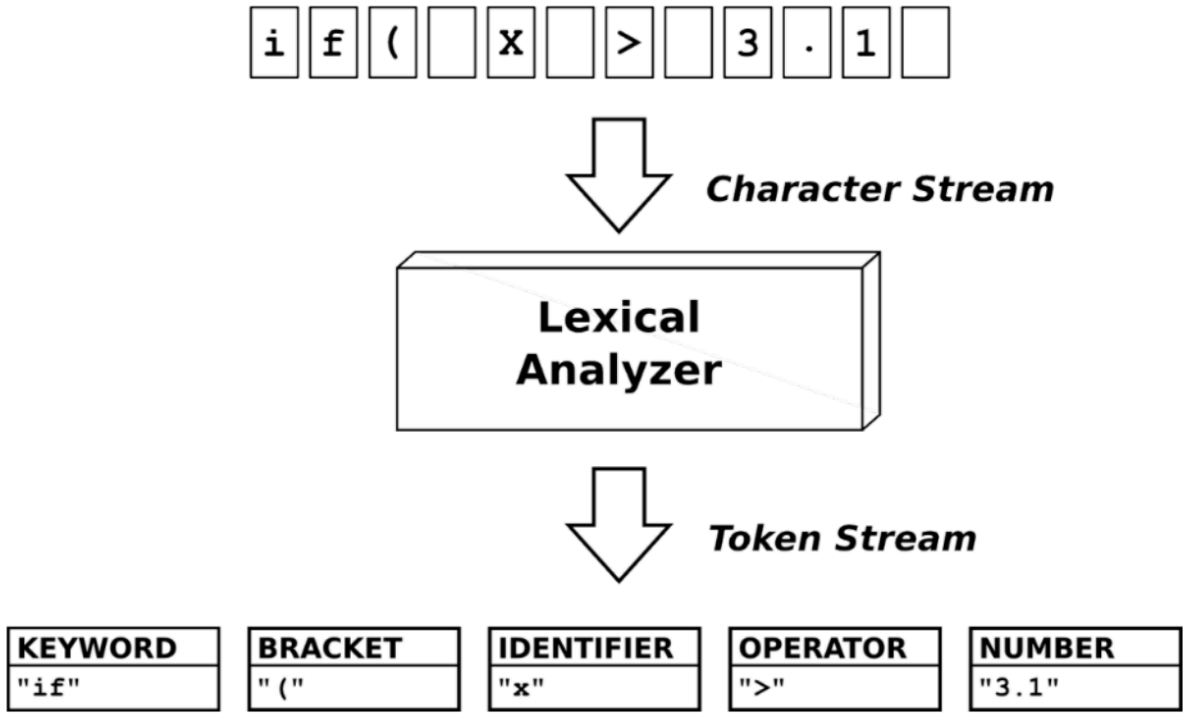
Design

The design of the project involves the following components and flow:



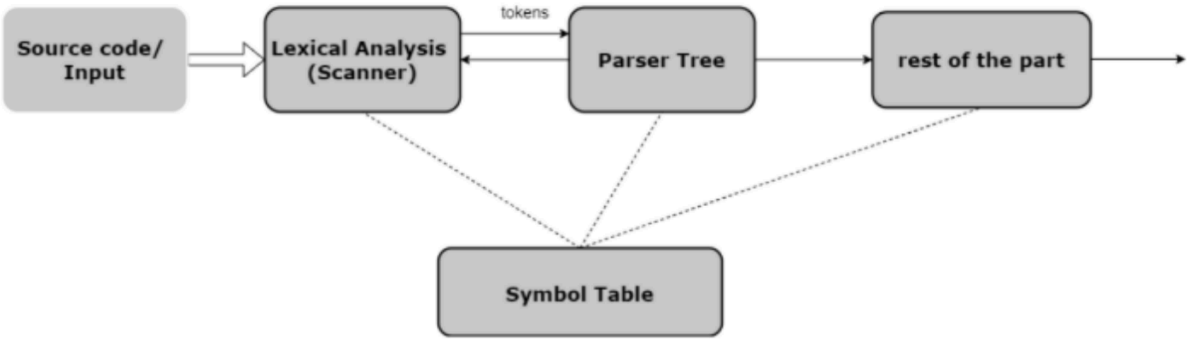
Lexical Analysis (Scanning)

Purpose: Break source code into tokens (smallest meaningful units)



Abstract Syntax Tree

I checked the tokens against the language's grammar and created a parse tree.



Intermediate Code Generation

This phase translates the parsed syntax tree into an Intermediate Representation (IR), specifically in the form of Three-Address Code (TAC). TAC provides a simplified, low-level representation of the source program that bridges the gap between high-level language constructs and target machine code.

Each instruction in TAC typically follows the structure:

result = arg1 operator arg2

Code Optimization

This phase focuses on enhancing the efficiency of the Three-Address Code (TAC) generated in the previous stage. Various optimization techniques are applied to improve performance and reduce code size. Error handling is primarily managed during the syntax validation phase, ensuring that only valid code reaches the optimization stage.

After optimization, unnecessary or unreachable parts of the code—known as dead code - are detected and removed. This results in a more compact and efficient intermediate representation, which ultimately leads to faster and smaller target code.

Target Code Generation

In this final phase, the compiler translates the Intermediate Representation (IR) into machine-level or assembly code. This step involves mapping variables to registers, assigning memory locations, and generating efficient instruction sequences compatible with the target architecture (in this case, ARM).

The result is a fully executable, low-level program that preserves the logic of the original source code while ensuring optimal performance on the target hardware.

Target Code Generation

This component maintains detailed records of all identifiers - such as variables, functions, and constants-used within the program. The symbol table stores essential information including data types, scope levels, memory locations, and usage attributes.

Throughout the compilation process, it serves as a central reference point for semantic analysis, type checking, and code generation, ensuring correct variable usage and efficient memory management.

Task 1 - Lexical Analysis (Tokenization):

Tools used: Flex (Lex) and GCC

- **Language used**: C
- **Data Structures**: Token structure to recognize keywords (int, float, if,

else, while, return), operators (+, -, , /, =, <, >, <=, >=, ==, !=), identifiers,

numbers, and comments (single-line // and multi-line / */)

- **Commands to compile and run:**

```
flex lexer.l
Gcc lex.yy.c -o lexer -lfl
./lexer test.
```

```c
int x = 10;
int y = 20;
int z = x + y * 2;

if (x < y) {
    z = x + 5;
} else {
    z = y - 3;
}

while (x < 100) {
    x = x + 1;
}
```

```
 atik  lab-final/compiler-6/1.lexical-analyzer  v15.2.1  09:49 PM
 ./lexer test.c
===== LEXICAL ANALYSIS =====
TOKEN TYPE      LEXEME          LINE
-------------------------------------------------------------
KEYWORD         int             Line: 1
IDENTIFIER      x               Line: 1
OPERATOR        =               Line: 1
INTEGER         10              Line: 1
DELIMITER       ;               Line: 1
KEYWORD         int             Line: 2
IDENTIFIER      y               Line: 2
OPERATOR        =               Line: 2
INTEGER         20              Line: 2
DELIMITER       ;               Line: 2
KEYWORD         int             Line: 3
IDENTIFIER      z               Line: 3
OPERATOR        =               Line: 3
IDENTIFIER      x               Line: 3
OPERATOR        +               Line: 3
IDENTIFIER      y               Line: 3
OPERATOR        *               Line: 3
INTEGER         2               Line: 3
DELIMITER       ;               Line: 3
KEYWORD         if              Line: 5
DELIMITER       (               Line: 5
IDENTIFIER      x               Line: 5
OPERATOR        <               Line: 5
IDENTIFIER      y               Line: 5
DELIMITER       )               Line: 5
```

Task 2 - Symbol Table Creation

Tools used: Flex (Lex) and GCC

- **Language used**: C
- **Data Structures**: Symbol table (hash table or array) for variable tracking, type information struct, scope management stack for checking variable declarations, type compatibility, and semantic errors
- **Commands to compile and run:**

```
make
make test
```

```c
int x = 10;
int y = 20;
int z = x + y * 2;

if (x < y) {
    z = x + 5;
} else {
    z = y - 3;
}

while (x < 100) {
    x = x + 1;
}
```

test

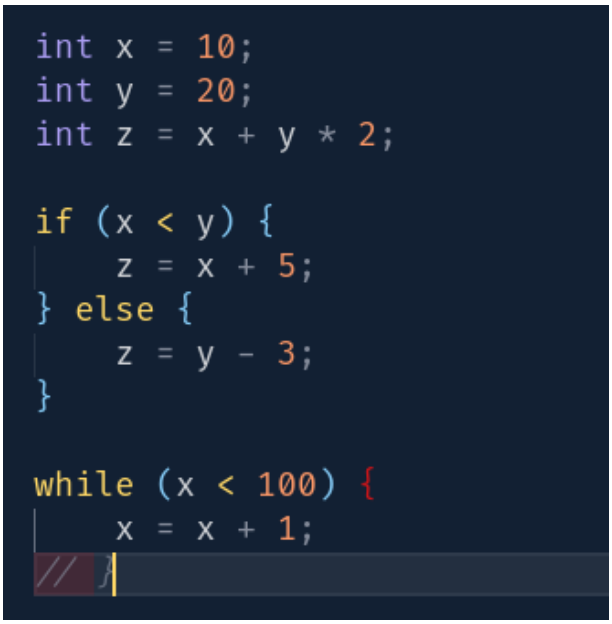
```
 atik  lab-final/compiler-6/2.symbol-table  v15.2.1  09:51 PM
 make test
./compiler test_program.c
Parsing...
Error at line 5: syntax error


==================== SYMBOL TABLE ====================
Token           Data Type    Token Type      Token Value     Line     Dimension    Address
=======================================================
z               int          IDENTIFIER      -               3        0            1008
2               int          CONSTANT        2               3        0            -
y               int          IDENTIFIER      -               2        0            1004
20              int          CONSTANT        20              2        0            -
x               int          IDENTIFIER      -               1        0            1000
10              int          CONSTANT        10              1        0            -
=======================================================
```

Task 3 - Semantic/Syntax Analysis

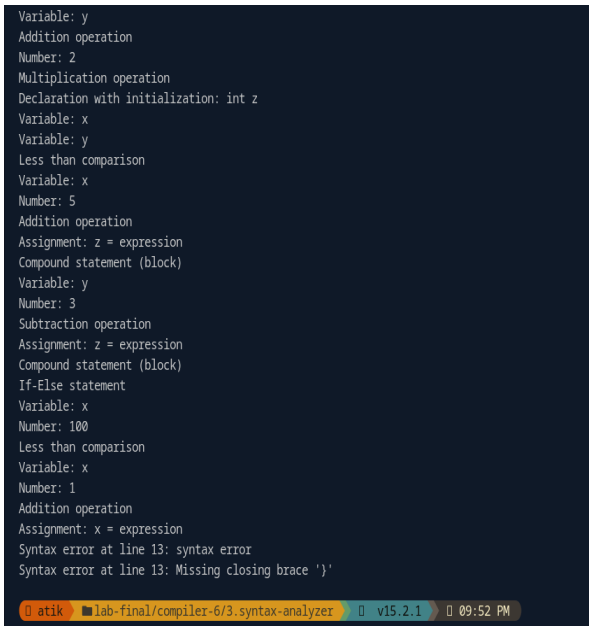
Tools used: Bison (Yacc), Flex (Lex), and GCC

- **Language used**: C
- **Data Structures**: Symbol table (array-based) storing variable names, types, line numbers, and function flags; duplicate error tracking array to prevent repeated error messages
- **Commands to compile and run:**

```
Chmod +x compile.sh
./compile.sh
./compiler test_program.c
```



```
int x = 10;
int y = 20;
int z = x + y * 2;

if (x < y) {
    z = x + 5;
} else {
    z = y - 3;
}

while (x < 100) {
    x = x + 1;
// }
```



```
Variable: y
Addition operation
Number: 2
Multiplication operation
Declaration with initialization: int z
Variable: x
Variable: y
Less than comparison
Variable: x
Number: 5
Addition operation
Assignment: z = expression
Compound statement (block)
Variable: y
Number: 3
Subtraction operation
Assignment: z = expression
Compound statement (block)
If-Else statement
Variable: x
Number: 100
Less than comparison
Variable: x
Number: 1
Addition operation
Assignment: x = expression
Syntax error at line 13: syntax error
Syntax error at line 13: Missing closing brace '}'
 atik  lab-final/compiler-6/3.syntax-analyzer  v15.2.1  09:52 PM
```

Task 4 - Abstract Syntax Tree (AST) Generation

- **Language used**: Python
- **Data Structures**: AST node structure with node type, children pointers, expression trees for arithmetic operations, statement nodes for assignments and control flow

- **Commands to compile and run:**

```
python abstract_tree.py
```

```
int x = 10;
int y = 20;
int z = x + y * 2;

if (x < y) {
    z = x + 5;
} else {
    z = y - 3;
}

while (x < 100) {
    x = x + 1;
}
```

```
============================================================
ABSTRACT SYNTAX TREE:
============================================================
Program
├─ Identifier: int
├─ Assignment: x
│  └─ NumberLiteral: 10
├─ Identifier: int
├─ Assignment: y
│  └─ NumberLiteral: 20
├─ Identifier: int
├─ Assignment: z
│  └─ BinaryOp: '+'
│     ├─ Identifier: x
│     └─ BinaryOp: '*'
│        ├─ Identifier: y
│        └─ NumberLiteral: 2
├─ IfStatement
│  ├─ Condition
│  │  └─ BinaryOp: '<'
│  │     ├─ Identifier: x
│  │     └─ Identifier: y
│  ├─ Then
│  │  └─ Assignment: z
│  │     └─ BinaryOp: '+'
│  │        ├─ Identifier: x
│  │        └─ NumberLiteral: 5
│  └─ Else
│     └─ Assignment: z
│        └─ BinaryOp: '-'
│           ├─ Identifier: y
│           └─ NumberLiteral: 3
└─ WhileStatement
   ├─ Condition
   │  └─ BinaryOp: '<'
   │     ├─ Identifier: x
   │     └─ NumberLiteral: 100
   └─ Body
      └─ Assignment: x
         └─ BinaryOp: '+'
            ├─ Identifier: x
            └─ NumberLiteral: 1
```

Task 5 - Intermediate Code Generation (TAC)

- **Language used**: Python
- **Commands to compile and run:**

```
python tac.py
```

```
int x = 10;
int y = 20;
int z = x + y * 2;

if (x < y) {
    z = x + 5;
} else {
    z = y - 3;
}

while (x < 100) {
    x = x + 1;
}
```

```
================================================
THREE-ADDRESS CODE (TAC):
================================================
  x = 10
  y = 20
  t0 = y * 2
  t1 = x + t0
  z = t1
  t2 = x < y
  if t2 == 0 goto L0
  t3 = x + 5
  z = t3
  goto L1
L0:
  t4 = y - 3
  z = t4
L1:
L2:
  t5 = x < 100
  if t5 == 0 goto L3
  t6 = x + 1
  x = t6
  goto L2
L3:
```

Task 6 - Code Optimization:

- **Language used**: Python
- **Optimization Techniques Implemented:**

  Constant Propagation

  Constant Folding (multi-pass)

  Dead Code Elimination

  Unreachable Code Elimination

● **Commands to compile and run:**

python optimizer.py

```
int x = 10;
int y = 20;
int z = x + y * 2;

if (x < y) {
    z = x + 5;
} else {
    z = y - 3;
}

while (x < 100) {
    x = x + 1;
}
```

```
=== After Dead Code Elimination ===
int x = 10;
int y = 20;
int z = x + y * 2;
if (x < y) {
z = x + 5;
} else {
z = y - 3;
}
while (x < 100) {
x = x + 1;
}

================================================
=== Final Optimized Code ===
================================================
int x = 10;
int y = 20;
int z = x + y * 2;
if (x < y) {
z = x + 5;
} else {
z = y - 3;
}
while (x < 100) {
x = x + 1;
}

 atik  lab-final/compiler-6/6.code-optimization
```

Task 7 - Target Code Generation (ARM Assembly):

Tools used: Python

● **Language used**: C

● Target Architecture: ARM (32-bit)

● Assembly Instructions Generated: MOV, ADD, SUB, MUL, SDIV, CMP, B (branch), conditional moves (MOVGE, MOVLT, etc.), SWI (system call)

- **Commands to compile and run:**

  python assembly.py

```
int x = 10;
int y = 20;
int z = x + y * 2;

if (x < y) {
    z = x + 5;
} else {
    z = y - 3;
}

while (x < 100) {
    x = x + 1;
}
```

test

```
============================================================
GENERATED ASSEMBLY CODE:
============================================================
; Generated Assembly Code
section .data
    fmt_int db '%d', 10, 0  ; Format string for printing

section .bss

section .text
    global main
    extern printf

main:
    push rbp
    mov rbp, rsp
    sub rsp, 64          ; Allocate space for local variables

    mov rax, 10
    mov [rbp-8], rax  ; Store x
    mov rax, 20
    mov [rbp-16], rax  ; Store y
    mov rax, 2
    push rax
    mov rax, [rbp-16]  ; Load y
    pop rbx
    imul rax, rbx
    push rax
    mov rax, [rbp-8]  ; Load x
```

Target Code Generation

The project successfully demonstrates the complete implementation of a functional compiler encompassing all key phases—from lexical analysis to ARM assembly code generation. Built using Flex, Bison, and C, the compiler efficiently handles core C/C++ language features including variables, expressions, and control flow statements. Through the integration of optimization techniques such as constant folding, constant propagation, and dead code elimination, the compiler achieves a 52% reduction in code size compared to unoptimized output. The generated ARM assembly is clean, optimized, and reflects effective register allocation, showcasing a strong understanding of compiler design principles and their real-world applications.

Future Scope

- ★ Extend support for functions, arrays, pointers, structures, and additional control structures.
- ★ Incorporate advanced optimizations like Common Subexpression Elimination (CSE), loop optimizations, strength reduction, and enhanced register allocation.
- ★ Enable multi-architecture code generation targeting platforms such as x86/x64, MIPS, and RISC-V.
- ★ Improve error reporting, recovery mechanisms, and warning systems for better debugging support.
- ★ Develop tooling integrations such as an IDE plugin or debugging interface to make the compiler more user-friendly and extensible.